

Lab 3: Control Flow for Data Cleaning

Topic: Loops, Functions, and Error Handling in R

Suggested Dataset: UCI Heart Disease Dataset
UCI Heart Disease Dataset

Relevant attributes: The dataset contains patient-related clinical variables including age, resting blood pressure (trestbps), cholesterol, maximum heart rate, etc.

Problem Statement:

A healthcare analytics team is analyzing patient cardiovascular data. Before statistical analysis, the dataset must be checked and cleaned because clinical data collected from different sources may contain invalid values, missing observations, and extreme measurements.

For this practical, use the **resting blood pressure (trestbps)** variable. To simulate realistic data-entry problems, deliberately introduce a few negative BP values, missing values, and extreme BP readings greater than 300 mmHg.

Tasks:

1. **Create a BP-cleaning function using if-else:**
 - Detect negative blood-pressure values and convert them to NA.
 - Detect systolic/resting BP values greater than 250 mmHg and cap them at 250.
 - Retain valid BP values without modification.
2. **Implement error handling using tryCatch():**
 - Safely calculate mean BP when missing values are present.
 - Calculate a suitable ratio, such as chol / trestbps.
 - Handle cases where the denominator is zero, NA, or invalid.
 - Generate an informative error/warning message instead of terminating the program.
3. **Compare loop-based and vectorized data cleaning:**
 - Use a for loop to identify invalid/outlier BP values.
 - Perform the same operation using vectorized R operations.
 - Measure execution time using system.time() or microbenchmark.
 - Compare the performance of the two approaches.
4. **Validate the cleaned data:**
 - Count missing BP values.
 - Find minimum, maximum, mean, and median BP.
 - Verify that no negative or >250 values remain after cleaning.

Expected Deliverables:

- cleaned_heart_data.csv
- Complete R script
- Custom BP-cleaning function
- tryCatch() error-handling function
- Loop vs. vectorized execution-time comparison
- Short conclusion explaining which method is more efficient

Learning Outcome:

Students will be able to **apply conditional statements, loops, functions, vectorized operations, and exception handling to clean a real-world healthcare dataset using R.**

Lab 4: Advanced Missing Data Handling

Topic: NA, NULL, NaN, Missing-Value Detection and Imputation

Suggested Dataset: UCI Adult / Census Income Dataset

[UCI Adult Dataset](#)

This dataset is particularly suitable because it contains a mixture of **numeric and categorical variables** such as age, work class, education, occupation, working hours, income, etc.

Problem Statement:

An organization is analyzing demographic and employment data for workforce analytics. However, the dataset contains incomplete and potentially invalid observations. Before performing statistical or predictive analysis, missing and inconsistent values must be systematically identified and treated.

Use the **UCI Adult dataset** for this experiment. After loading the dataset, deliberately introduce a small number of NA, blank strings (""), NaN, and impossible numeric values such as age = 999 so that students can examine different missing-data scenarios.

Important R concept: NULL normally represents the absence of an R object rather than a value inside a data-frame cell. Therefore, blank strings should not technically be described as NULL. This lab should teach students the distinction between NA, NULL, NaN, and "".

Tasks:

1. **Identify different forms of missing or invalid data:**
 - Use is.na() to identify NA.
 - Use is.nan() to identify NaN.
 - Demonstrate is.null() using an R object and explain why NULL behaves differently inside a data frame.
 - Identify blank strings using x == "".
 - Identify impossible values such as age = 999.

- Generate variable-wise missing-value summaries using `naniar::miss_var_summary()`.
2. **Develop an appropriate missing-data treatment strategy:**
 - Convert impossible numeric values to NA.
 - Replace blank categorical values with "Unknown".
 - Impute missing numeric values using the median.
 - Remove observations containing unrecoverable NaN values where appropriate.
 - Use `complete.cases()` to identify complete/incomplete observations.
 3. **Create a custom imputation function:**
 - Accept a numeric vector as input.
 - Identify missing values.
 - Calculate the median of valid observations.
 - Replace missing observations with the median.
 - Return the cleaned vector.
 4. **Analyze missingness before and after cleaning:**
 - Calculate number and percentage of missing values.
 - Use `naniar` to visualize missingness patterns.
 - Compare missingness before and after treatment.
 5. **Validate the cleaned dataset:**
 - Use `skimr::skim()` for a comprehensive summary.
 - Verify that impossible age values have been removed.
 - Verify that selected numeric variables no longer contain untreated missing values.
 - Verify that blank categorical values have been replaced.

Expected Deliverables:

- `cleaned_adult_data.csv`
- R script containing missing-value detection and treatment
- Custom median-imputation function
- Before/after missing-data summary
- Missingness visualization
- `skimr::skim()` validation output
- Short interpretation of the cleaning results

Learning Outcome:

Students will be able to **differentiate between NA, NULL, NaN, and blank values in R**

and implement appropriate detection, imputation, removal, visualization, and validation techniques on a real-world dataset.

- SUBMIT COLAB NOTEBOOK PDF
- ALSO UPLOAD THE SUBMISSION ON GIT ACCOUNT AND SHARE LINK OF GIT UPLOAD IN SUBMISSION

I hereby declare that:

1. The assignment / examination has been designed in alignment with the approved curriculum and intended Learning / Course Outcomes.
2. The assessment incorporates the following key elements:
 - o Problem-solving ability
 - o Critical reasoning
3. An AI tool has been used only for reviewing and enhancing assessment quality (where applicable) and not as a replacement for academic judgment.
4. The assignment has been reviewed, finalized, and approved by me, and I take full academic responsibility for its content, rigor, fairness, and integrity.
5. I understand that submission of this declaration is mandatory for all assignments and that it must be presented during the Meeting with the Chief Quality Officer (CQO) or any quality audit.

AI Feedback Scores

Rating: 1 – Lowest | 4 – Highest

Parameter	Score
Real-life Application	4 / 4
Problem-solving Ability	4 / 4
Critical Reasoning	4 / 4
Creativity	4 / 4
Alignment with Learning / Course Outcomes	4 / 4

Estimated Time Required

1 Hour 45 Minutes – 2 Hours

Lab 3: 50–55 Minutes

- Develop reusable functions using if-else
- Implement tryCatch() for exception handling
- Clean invalid values, outliers, and missing entries

- Compare loop-based and vectorized approaches for data processing
- Generate benchmark results and document the developed functions

Lab 4: 55–65 Minutes

- Identify different types of missing data using built-in and external R packages
- Apply suitable imputation and data cleaning strategies
- Validate the cleaned dataset using **skimr**
- Generate missing data reports and visualize missingness patterns

COs Linked with the Assignment

CO1, CO2, CO3

LOs Linked with the Assignment

LO 2.1, LO 2.2, LO 2.3, LO 2.4, LO 3.1, LO 3.2

Justification

This laboratory assignment provides students with practical exposure to one of the most critical stages of the data science lifecycle—**data cleaning, preprocessing, and robust data handling**. The laboratory exercises are based on realistic healthcare and employee survey datasets that simulate challenges commonly encountered by data analysts and data scientists in real-world applications. Students are required to identify invalid observations, detect outliers, manage multiple forms of missing data, implement structured error handling, and validate the quality of cleaned datasets using widely adopted R packages.

The third laboratory develops programming proficiency by requiring students to design reusable functions using conditional statements, implement exception handling through **tryCatch()**, and compare conventional iterative solutions with efficient vectorized implementations. These activities strengthen algorithmic thinking, debugging skills, code optimization, and computational efficiency while encouraging students to evaluate different implementation strategies.

The fourth laboratory extends these concepts by introducing advanced missing data management techniques. Students distinguish between **NA, NULL, and NaN**, apply appropriate imputation methods, validate processed datasets using **skimr**, and visualize missingness patterns to assess data quality. Such activities closely reflect industry-standard data preprocessing workflows that precede statistical analysis and machine learning model development.

The assignment demonstrates strong alignment with **CO1** by reinforcing fundamental R programming concepts, functions, control structures, and data handling techniques. It strongly supports **CO2** through practical implementation of data manipulation, preprocessing, and analytical reasoning using modern R packages. Furthermore, it substantially contributes to **CO3** by requiring students to import, clean, transform, validate, and prepare datasets for subsequent statistical analysis and machine learning tasks. The inclusion of benchmarking, reusable

function development, comprehensive documentation, and reproducible programming practices further enhances students' professional coding skills and critical thinking abilities. Overall, the assignment effectively bridges theoretical programming concepts with practical data engineering techniques, providing a strong foundation for advanced statistical computing and machine learning modules.

AI Tool Used

ChatGPT

Prompt(s) Used

"Evaluate the given R Programming laboratory assignment using a 1–4 rubric for Real-life Application, Problem-solving Ability, Critical Reasoning, Creativity, and Alignment with Learning / Course Outcomes. Identify the estimated time required, map the relevant Course Outcomes (CO1–CO6) and Learning Outcomes (LOs), and justify the ratings based on the R Programming syllabus covering control flow, functions, exception handling, vectorized programming, missing data handling, data preprocessing, validation, and data quality assessment."